**Word Representations —** *One-hot-encoding* issues: 1. orthogonal definitions; every word is equidistant from every other word, 2. high-dimensionality (and sparse), 3. fixed vocabulary; must use `UNK` token to represent unknown words. *WordNet*: a large, manually-curated database grouping synonyms into synsets with definitions and usage examples. Downsides: manual, no rare/new meanings, vectors still orthogonal & equidistan, homonym disambiguation. *Distributional Hypothesis*: similar words occur in similar contexts. *Word2Vec*: learn embeddings. Architectures: CBOW & skip-gram. Training strategy: hierarchical softmax & negative sampling. *CBOW*: predict $w_t$ based on surrounding context words. *Skip-gram*: predict context words based on $w_t$. The model is a neural network with one hidden layer: $p(w_{t+j} \mid w_t) = \mathrm{softmax}(xW_{\text{in}}W_{\text{out}})$, where $x$ is the one-hot encoded target word and the output is a probability distribution over the vocabulary that captures $p(w_{\text{context}} \mid w_{\text{target}})$. In skip-gram we have several outputs for each context word. The matrix $W_{\text{in}}$ maps the input to the embedding for that target word. The loss function for skip-gram is $\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\ne 0}\log p(w_{t+j}|w_t;\theta)$. *Negative sampling*: vocabularies could be large in size, making softmax calculation expensive. Instead we can use binary classification to distinguish context words. We take random negative samples that aren't in the context. Then maximise the prob. that positive samples are predicted true and negative samples are predicted false. We pick $k$ negative samples, $k \in [5, 20]$. The model now has $|V|$ classifiers, defined by sigmoids. Frequency sampling better than random: $p(w_i) = \frac{f(w_i)^{\frac{3}{4}}}{\sum_{w'} f(w')^{\frac{3}{4}}}$.

Downsides to word embeddings: need large data to train, low quality for rare words, antonyms have similar distributions, cannot capture different meanings of a word. *Byte-pair encoding*: 1. Start with vocabulary of individual characters. 2. Find which vocabulary pair occurs most frequently. 3. Add that combination as a new vocabulary item. 4. Merge all such occurrences in the corpus. 5. Repeat until desired number of merges (hyperparameter) have been performed. We append each word with an end-of-word symbol _, because it distinguishes suffixes and tells us where to insert spaces when reassembling text. *TF-IDF*: Term Frequency Inverse Document Frequency $\mathrm{TF}_{w,d} = \frac{\mathrm{count}(w,d)}{\sum_{w'}\mathrm{count}(w',d)}$ $\mathrm{IDF}_{w,D} = \log\frac{|D|}{|\{d\in D: w\in d\}|}$. Upweights words important in a document, downweights words that appear everywhere.

**Classification —** Bayes' Rule: $P(y|x) = \frac{P(x|y)P(y)}{P(x)}$. Because $P(x)$ is fixed, $\hat{y} = \mathrm{argmax}_y P(y|x) = \mathrm{argmax}_y P(x|y)P(y)$. *Add-one smoothing*: multiplication with zero probabilities collapse the final result to 0. This can happen if we encounter an unseen word. $P(x_i|y) = \frac{\mathrm{count}(x_i,y)+1}{\sum_{x\in V}\mathrm{count}(x,y)+1}$ Can take too much probability mass from likely occurrences, so try +$k$ smoothing with a smaller $k$. *Binary Naive Bayes*: ignore duplicates of a feature in each instance. *Controlling for negation*: append `NOT_` after any logical negation until the next punctuation mark. Naive Bayes problems: all features are considered equally important, conditional independence is an assumption, context of words not considered, new words cannot be used. *Cross-Entropy Loss*: measures how close predicted distribution $Q$ is to actual distribution $P$; $H(P,Q) = -\sum_i P(y_i)\log Q(y_i)$. *Sigmoid*: $\frac{1}{1+e^{-x}}$. *Softmax*: $g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

*Document representation*: variable-size documents can be represented as the average of their embeddings, but this loses positional information. *RNNs*: process sequential data. $h_{t+1} = \tanh(Wh_t + Ux_t)$. The output at time $t$ is $y_t = W_{hy}h_t + B_y$. Limitations: vanishing gradient problem. $\tanh$ and $\mathrm{sigmoid}$ derivatives are between 0 and 1, and 0 and 0.25. Repeated multiplication means gradients disappear, so model is less able to learn from earlier inputs. Also, gradient for earlier layers involve repeated multiplication of the same weight matrix $W$. Depending on the dominant eigenvalue, this can cause it to vanish or explode. *CNNs*: filter width = embedding dimension, filter height = 2 to 5 tokens. CNNs perform well if the task involves key phrase recognition. RNNs better understand longer-range dependencies.

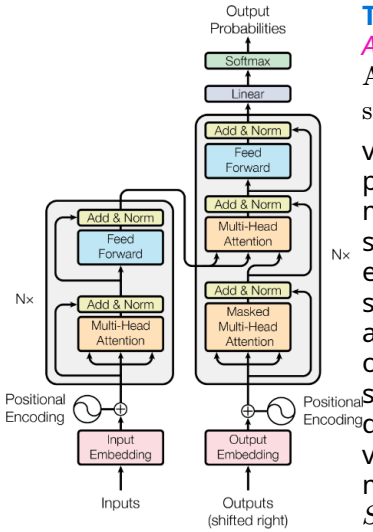
**Language Models —** *N-gram modelling*: find the most probable next word using $P(w_n \mid w_1^{n-1})$ by counting occurences. $N$-gram approximation: $\approx P(w_n \mid w_{n-N+1}^{n-1})$ *Perplexity)* = $P(w_1, w_2, ..., w_n)^{-\frac{1}{n}}$. We aim to minimise perplexity. Perplexity = $e^H$, where $H$ is CE loss. *Extrinsic* evaluation: choose LM that works best on downstream task. Perplexity is *intrinsic*.

**Neural Language Models —** *Backoff Smoothing*: $P(w_i|w_k^{i-1}) = 0$, use $0.4P(w_i|w_{k+1}^{i-1})$ recursively instead. *Interpolation*: combine evidence from different n-grams: $\lambda_1 P(w_i \text{ trigram}) + \lambda_2 P(w_i \text{ bigram}) + \lambda_3 P(w_i)$ where $\sum_i \lambda_i = 1$ *Bidirectional RNNs*: concatenate hidden states of a forward and backward RNN. This allows the model to see into the future. *Multi-layered RNNs*: hidden states from one layer become inputs of another. *LSTMs*: cell states provide long-term memory that helps against vanishing/exploding gradient because (1) additive formula → no repeated multiplication and (2) forget gate allows model to choose when to preserve and vanish gradients. $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and $h_t = o_t \odot \tanh c_t$, where $g_t = \tanh(Wx_t + Wh_{t-1} + b)$ and $i_t, f_t, o_t = \sigma(Wx_t + Wh_{t-1} + b)$. *GRUs*: reset and update gate, no cell state. $g_t = \tanh(Wx_t + r_t \odot (Wh_{t-1}) + b)$ and $h_t = (1 - z_t) \odot g_t + z_t \odot h_{t-1}$.

**Machine Translation —** *Statistical MT*: Uses a parallel dataset. Model $p(t|s)$ using Bayes' Theorem. Maximise $p(t)p(s|t)$ - $p(t)$ is the *language model*, $p(s|t)$ is the *translation model*. Downsides: sentence alignment, word alignment, statistical anomalies, idioms, out-of-vocab words. *RNNs*: bidirectional encoder which feeds into a unidirectional encoder. Encoded representation is context vector $c$ - could be final hidden state of encoder, or mean of all hidden states. Use *teacher forcing* to avoid accumulation of errors - can do at every time-step, or a proportion of time-steps. Loss function: $-\sum_{t=1}^{T}\log p(\hat{y}_t|y_{<t}, c)$ where $\hat{y}$ are decoder outputs. Downsides: encoder representation has fixed capacity; gets worse for longer sentences. *Attention*: additive/MLP, multiplicative, self-attention. MLP: $c_t = \sum_{i=1}^{I}\alpha_i h_i$. $\alpha_i$ comes from *alignment function*: $e_i = v^\top \tanh(Ws_{t-1} + Uh_i)$ where $s$ is previous decoder hidden state. $v, W, U$ are learnable. Compute $e_i \in \mathbb{R}$ for every encoder hidden state, then normalise using softmax to get $\alpha_i$ scores.

*BLEU score*: $\text{BLEU-4} = \mathrm{BP}\left(\prod_n^4 p_n\right)^{\frac{1}{4}}$, modified precision $p_n = \frac{\text{total unique overlap}_n}{\text{total } n\text{-grams}}$, BP = brevity penalty, e.g. $\min\left(1, \frac{c}{r}\right)$ where $c$ = candidate length, $r$ = reference length. Good: fast/cheap, language-independent, corpus-level correlation. Bad: no synonyms, no recall, no grammar/fluency, unreliable at sentence-level. *Chr-F*: character n-gram $F_\beta$ score, character-level handles morphologically rich languages. *TER*: translation error rate = Number of edits / number of words in reference. *ROUGE*: evaluating summaries. ROUGE-$n$ is $F$-score of $n$-gram counts over references and candidates. ROUGE-$L$ is $F$-score of longest common subsequence. *METEOR*: $m$ unigrams matched by (1) exact match, (2) stem match, (3) synonym match. Precision and recall calculated from these matches. $F_{\text{mean}} = \frac{10PR}{9P+R}$. Unigrams grouped into least nº of chunks, $\mathrm{ch}$. Penalty = $\gamma\left(\frac{\mathrm{ch}}{m}\right)^\beta$. $\mathrm{METEOR} = F_{\text{mean}}(1 - \mathrm{Pen})$ *BertScore*: convert candidate and reference to BERT embeddings, then pairwise cosine similarity of each token. *Greedy decoding*: output most likely word at each timestep. Fast, but doesn't look into future. *Beam search*: keep $k$ most likely paths at each timestep. *Temperature sampling*: change distribution using temperature param.

Divide logits by $T$, then apply softmax and sample distribution. Low temperature $\rightarrow$ peaky dist., close to deterministic. High $T \rightarrow$ close to uniform dist. *Back translation*: train a model translating in reverse direction. Use outputs of that model to train final model. Adds noise. *Batching/padding rules*: group similar-length sentences together in a batch, train model on smaller/ simpler sentences first.




**Transformers —** *Self Attention*: $\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$. Large values of $d_k$ yields dot products large in magnitude, pushing softmax into regions with extremely small gradients, so a scaling factor is applied. The output is a set of values, weighted by similarity between keys and queries. Queries, keys and values are learnable weight matrices. $V \in \mathbb{R}^{S \times d_h}$ where $S$ is sentence length, $d_h$ is attention vector dimension. *Multi-head self attention*: perform self-attention head $n$ times. We simply concatenate each head's output before applying the output projection. *Layer normalisation*: normalises the features of **one** sample in **one** batch. $B = \{x_1, ..., x_N\} \mapsto \{\mathrm{LN}(\hat{x}_1), ..., \mathrm{LN}(\hat{x}_N)\}$ where $\hat{x} = \frac{x-\mu}{\sigma}$ and $\mathrm{LN}(x) = \gamma \odot x + \beta$; $x_n, \gamma, \beta \in \mathbb{R}^d$ *Residual connections*: shortcut for information flow. Prevent vanishing gradient, network learns $\delta$. Layer output is $F(x) + x$ instead of $F(x)$. *Position-wise FNN*: all tokens independently pass through the same two-layer network. $\mathrm{FNN}(x) = \max(0, xW_1 + b)W_2 + b_2$. This is where knowledge is stored. *Positional encoding*: returns $\mathrm{embedding} + \mathrm{pos_enc}_i$. $\mathrm{PE}_{\mathrm{pos},2i} = \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$ and $\mathrm{PE}_{\mathrm{pos},2i+1} = \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d}}}\right)$ where $\mathrm{pos}$ is the position of the word and $i$ is the dimension index. Because $\sin(a+b)$ is a linear combination of $\sin(a)$ and $\cos(b)$, this allows the model to attend to relative positions. Cosine distance of positional encodings decreases as relative distance increases. *Masking*: do not attend to future tokens. Before softmax set illegal connections in $QK^\top$ matrix to $-\infty$ so model cannot attend. *Cross attention*: when decoding a token, use attention to look at encoded words. Use key, value tensors from last encoder layer and send them to all decoder layers. Queries come from decoder. *Transformer training loop*: create source & target mask, run encoder, run decoder, output logits for token prediction. *Weight tying*: embedding matrix and output projection matrix in decoder are shared.

**Pre-training Models —** *ELMo*: multi-layered bi-directional LSTM. Predict word at each stage. Concatenate representations from both directions. ELMo can be concatenated to an existing embedding layer. *BERT*: encoder-only transformer model. Outputs encoding of input tokens. Primary BERT objective is *masked language modelling*: transform 15% of words. 80% replace with `[MASK]` token, 10% random word, 10% stay the same. Random words encourage the model to use context and improves robustness. Secondary pre-training objective is *next sentence prediction*: given two sentences, predict if they appeared in that order in the source. *Using BERT*: remove masked LM head. (1) use BERT to find representation vectors for downstream task. (2) Put minimal neural architecture on top of BERT & train the whole thing. (*fine-tuning*). *Classification*: add `[CLS]` token in input that represents whole sentence. Encoded representation is class label. Sentences can be separated by `[SEP]` token. *SpanBERT*: mask contiguous tokens *Sparse attention*: standard self-attention is $O(N^2)$. Global tokens are designated to attend to every other token, e.g. `[CLS]`. *Big bird* uses random, window (local) and global attention. *ERNIE*: improve understanding by integrating **knowledge graphs**. Model takes entity inputs. This is combined with tokens in a fusion layer. *ImageBERT*: encodes detected objects as additional tokens. Training objectives: masked language modelling, masked object classification, masked object feature prediction, classifying image & text relevancy. *ViLBERT*: two parallel BERT-like encoders (text & image) sharing key/value matrices through co-attention. *Encoder-decoder*: possible bjectives: prefix language modelling, sentence permutation deshuffling, BERT-style token masking, replace corrupted spans. *Instructional training*: prefix input with string describing task. Can generate training data using templates, then train on tasks phrased as instructions. *Decoders*: can use *fine tuning* layer, *zero-shot* (task as natural language), *one-shot* (zero-shot with example), *few-shot*. We can use *step-by-step guiding*, *chain-of-thought*. *Retrieval-Augmented Generation (RAG)*: offload knowledge. Limitations of instruction fine-tuning: some tasks are open-ended and some mistakes are worse than others. Solutions: reinforcement learning with human feedback (pairwise comparisons).

**PoS Tagging —** useful for NER, pre-processing, semantic parsing. *Open class tag*: words with meaning. *Closed class tags*: functional words, no new additions. *HMM*: infer states (tags) from observations (words) with transition probabilities, emission probabilities and init state. Assumptions: **Markov**; future tag depends only on current state (bigram of tags), **independence**: prob. of a word depends on current state, not on prev. words. $P(T|W)P(W) \approx P(t_1)P(w_1|t_1)P(t_2|t_1)...P(t_n|t_{n-1})P(w_n|t_n)$ *Naive*: take max. $P(t_i|t_{i-1})P(w_i|t_i)$ at each step. *Viterbi*: find overall best path. Track probability of being in state $j$ after $t$ observations, for every state. Recursive: $v_t(j) = \max_{i=1}^{N} v_{t-1}(i)P(q_j|q_i)P(o_t|q_j)$. Alternative: *beam search*. *MEMM*: use logistic regression classifier to model $P(t_i)$ using many features: $w_i$, $t_i$, word shape etc. Viterbi for MEMM: $v_t(j) = \max_{i=1}^{N} v_{t-1}(i)P(q_j|q_i, o_t)$ *RNN*: input word embeddings, output softmaxed tag probabilities.

**Constituency Parsing —** *CKY*: change grammar to CNF ($X \rightarrow YZ$ or $X \rightarrow w$). $O(n^3 \mid |G|)$ $n$ is length of string, $|G|$ is size of CNF grammar. Cell `[i,j]` contains set of **non-terminals** that represent span i to j of input. This does not scale: too many possible parse trees in a comprehensive grammar. *Statistical parsing*: find most likely parse. Learn with *tree banks*. Augment grammar with probability of each rule. Pick most probable parse tree: $\mathrm{argmax}_{t \in \mathcal{T}(s)} P(t)$ *Probabilistic CKY*: dynamic table $\pi[i, j, X]$ = max prob. of rule $X$ spanning $i...j$. Final result is $\pi[1, n, S]$. Issues: **independence** assump. (word $\leftarrow$ tag), no lexical conditioning. Fixes: split non-terminals into types, lexicalised PCFG. *Precision*: nº correct constituents $\in$ hyp ÷ total nº constituents $\in$ hyp. *Recall*: nº correct constituents $\in$ hyp ÷ total nº constituents in ref. *Lexicalised PCFG*: each rule has a head. Use affinity between head words.

**Dependency Parsing —** connect head to dependent. Advantages: languages with free word order. Form a **tree**. *Projective*: no crossing dependency arcs when words laid out in linear order. $\forall(\mathrm{head}, \mathrm{dep})\exists$ path from head to every word betewen head & dep. *MaltParser*: stack σ, buffer β, arcs. Actions: shift, left arc, right arc. Each action predicted by ML classifier. *Acc/Prec/Rec*: use overlapping dependencies. *Neural parsing*: encoder-decoder translation to linearised tree.